



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CARRERA INGENIERÍA ELECTRÓNICA

**PROYECTO INTEGRADOR PARA LA OBTENCIÓN DEL
TÍTULO DE GRADO INGENIERO ELECTRÓNICO**

**“DISEÑO, DESARROLLO E IMPLEMENTACIÓN DE UN
SOFTWARE PARA LA ADQUISICIÓN Y ANÁLISIS DE ONDAS
DE IMPULSO TIPO RAYO (1,2/50 μ s).”**

Alumno

Tardón, Damián David

Director

Ing. Blasco, Marcos Javier

Co-Director

Ing Serra, Gabriel Horacio

Córdoba, República Argentina

Abril / 2026

Capítulo 1

Implementación del motor de procesamiento (DSP)

1.1 Algoritmo de cálculo de parámetros

El motor de cálculo implementado determina los parámetros característicos de la onda de impulso atmosférico tipo rayo (1, 2/50 μ s) bajo los lineamientos de la norma IEC 60060-1. El procesamiento se ejecuta sobre la curva de tensión de ensayo $u_t(t)$, definida como la suma de la curva base $u_m(t)$ y la curva residual filtrada $r_f(t)$.

El valor cresta de la tensión de ensayo (U_t) se determina como el máximo valor absoluto del vector resultante $u_t(t)$, conservando el signo correspondiente a la polaridad del impulso analizado.

$$U_t = \max(|u_m(t) + r_f(t)|) \cdot \text{sgn}(U_e) \quad (1.1)$$

Donde:

U_t : valor cresta de la tensión de ensayo.

$u_m(t)$: vector de la curva base ajustada.

$r_f(t)$: vector de la curva residual filtrada.

U_e : valor extremo de la curva registrada compensada en *offset*.

La norma define el tiempo de frente (T_1) a partir de una recta virtual que cruza los puntos correspondientes al 30 % y 90 % del valor cresta en la zona de ascenso de la señal.

$$T_1 = \frac{t_{90} - t_{30}}{0,6} \quad (1.2)$$

Donde:

T_1 : tiempo de frente.

t_{90} : instante de tiempo en el que la tensión alcanza el 90 % de U_t .

t_{30} : instante de tiempo en el que la tensión alcanza el 30 % de U_t .

Para establecer el origen temporal de la onda, se calcula el origen virtual (O_1), proyectando la recta definida anteriormente hasta su intersección con el eje de abscisas ($U = 0$).

$$O_1 = t_{30} - 0,5 \cdot (t_{90} - t_{30}) \quad (1.3)$$

Donde:

O_1 : origen virtual de la onda de impulso.

El tiempo de cola (T_2) se define como el intervalo temporal entre el origen virtual y el instante en que la tensión en la cola de la onda decrece al 50 % del valor cresta.

$$T_2 = t_{50} - O_1 \quad (1.4)$$

Donde:

T_2 : tiempo de cola.

t_{50} : instante de tiempo donde la amplitud de la señal decrece al 50 % de U_t .

La sobreoscilación de alta frecuencia en el pico de la señal (*overshoot*), si se encuentra presente, se cuantifica relacionando el pico de la curva de tensión de ensayo con el máximo de la curva base.

$$OS = \left(\frac{U_e - U_b}{U_e} \right) \cdot 100 \% \quad (1.5)$$

Donde:

OS : magnitud de la sobreoscilación porcentual.

U_e : valor pico absoluto de la curva de tensión de ensayo.

U_b : valor máximo absoluto de la curva base.

1.2 Filtrado de señales adquiridas

El tratamiento de los datos crudos inicia con la remoción del *offset* de corriente continua propio del sistema de medición. El algoritmo extrae un bloque de muestras previas al evento de disparo (*pre-trigger*) para calcular la media aritmética del ruido de fondo, restando esta constante a la totalidad de las muestras de la curva registrada.

Para la remoción del ruido electromagnético de alta frecuencia inherente al ensayo, se implementa un filtro digital de respuesta impulsional infinita (IIR) con coeficientes calculados paramétricamente según las indicaciones que prescribe la norma IEC 60060-1 [X]. La función de transferencia del filtro está definida por la siguiente

ecuación de diferencias:

$$y[i] = b_0 \cdot x[i] + b_1 \cdot x[i - 1] + a_1 \cdot y[i - 1] \quad (1.6)$$

$$\begin{aligned} b_0 &= b_1 = \frac{c}{1 + c} \\ a_1 &= \frac{1 - c}{1 + c} \\ c &= \tan\left(\frac{\pi \cdot T_S}{\sqrt{d}}\right) \end{aligned}$$

Donde:

$$d = 2,2 \times 10^{-12}$$

T_S : Intervalo de muestreo usado al registrar la señal.

$x[i]$: Arreglo de muestras de entrada al filtro.

$y[i]$: Arreglo de muestras de salida del filtro.

A fines de evitar la distorsión del retardo de grupo (fase no lineal) que introducen los filtros IIR estándar, se emplea la técnica de filtrado de fase cero mediante la función `scipy.signal.filtfilt` de la biblioteca SciPy. Este método procesa el vector de la curva residual bidireccionalmente (hacia adelante y hacia atrás), anulando el corrimiento de fase temporal y garantizando la fidelidad morfológica de la onda.

1.3 Interpolación de datos

Como se vio anteriormente, el osciloscopio digital discretiza verticalmente la señal. Esta cuantización, sumada a la frecuencia de muestreo finita, imposibilita que los umbrales de tensión (30 %, 50 % y 90 %) coincidan exactamente con las muestras discretas de la matriz extraída. Para sortear esta limitación, el motor DSP implementa un algoritmo de interpolación lineal que aproxima el cruce de umbrales entre dos muestras subyacentes.

$$t_x = t_1 + (V_x - V_1) \cdot \frac{t_2 - t_1}{V_2 - V_1} \quad (1.7)$$

Donde:

t_x : instante de tiempo interpolado correspondiente al umbral de tensión buscado (V_x).

(t_1, V_1) y (t_2, V_2) : coordenadas de tiempo y tensión de las muestras discretas inmediatamente anterior y posterior al umbral V_x .

INSERTAR IMAGEN PARA MOSTRAR EL CONCEPTO DE INTERPOLACION.

1.4 Validación algorítmica con patrón TDG (IEC 61083-2)

La validación del motor matemático se realiza inyectando vectores de datos de prueba, generados por el *software Test Data Generator* certificado por la norma IEC 61083-2, que establece los requisitos de los *software* para ensayos de impulso de tensión y corriente.

El programa procesa estos archivos de datos estandarizados, ejecutando de forma ciega las rutinas de, corrección de *offset*, ajuste de curva, filtrado bidireccional, interpolación y detección de umbrales. Luego, los parámetros calculados en cada onda (U_t , T_1 , T_2 y OS) se contrastan contra los valores nominales y los límites de aceptación establecidos por la norma.

Para certificar el correcto funcionamiento de la implementación DSP en Python, el grupo de ondas analizadas debe arrojar un error relativo porcentual inferior a los límites indicados para ese grupo.

1.5 Descripción funcional del software de procesamiento

1.5.1 Clase **LightningImpulseAnalyzer**

El motor de procesamiento digital de señales (DSP) del *software* está constituido por la clase **LightningImpulseAnalyzer**. Su función principal es encapsular el vector unidimensional de *raw data* adquirido por el osciloscopio, y ejecutar secuencialmente las rutinas matemáticas para la determinación de los parámetros del impulso según la norma IEC 60060-1, cumpliendo con la validación de la norma IEC 61083-2. Internamente maneja atributos de estado para almacenar los vectores procesados en cada etapa (señal sin *offset*, curva base, curva residual, curva residual filtrada y curva de ensayo), así como los parámetros de ajuste de la regresión no lineal, y el diccionario contenedor de los resultados finales.

- Define la estructura de datos que soporta las operaciones vectorizadas mediante la biblioteca **NumPy**.
- Gestiona la memoria almacenando vectores intermedios para evitar el recálculo durante la ejecución secuencial.
- Orquesta la ejecución de tuberías de procesamiento (*pipelines*) diferenciadas según el tipo de impulso (completo o cortado en la cola).

Método `__init__`

Inicializa la instancia de análisis, configurando el entorno matemático y validando las magnitudes de entrada fundamentales para la discretización temporal.

1. Ejecuta validaciones de control sobre el periodo de muestreo y el factor de peso de la regresión (`sigma_fit`), lanzando excepciones si los valores son inválidos.
2. Convierte la lista de datos crudos en un vector de tipo `numpy.ndarray`.
3. Construye el eje temporal discretizado (`time_axis`) multiplicando los índices del vector por el periodo de muestreo.
4. Declara e inicializa en un estado nulo (`None`) los atributos de clase y el diccionario de resultados.

Método `_remove_offset`

Remueve la componente de corriente continua (ruido de fondo u *offset*) de la señal registrada mediante el análisis probabilístico de la ventana de *pre-trigger*.

```
def _remove_offset(self):
    self.idx_peak = np.argmax(np.abs(self.raw_voltage))
    front = self.raw_voltage[:self.idx_peak]
    n = int(self.idx_peak * 0.3)
    if n < 1:
        raise ValueError("Error: ...")
    pre_trigger = self.raw_voltage[:n]
    mean = np.mean(pre_trigger)
    std = np.std(pre_trigger)
    diff = np.abs(front - mean)
    max_index = np.flatnonzero(diff >= 5 * std)[0]
    diff_2 = np.abs(front[:max_index] - mean)
    valid_index = np.flatnonzero(diff_2 <= std)
    if valid_index.size > 0:
        idx = valid_index[-1]
        background_noise = self.raw_voltage[:idx]
    else:
        idx = 0
        background_noise = self.raw_voltage[0:1]
    self.time_star_impulse = self.time_axis[idx]
    self.offset_value = np.mean(background_noise)
    self.zeroed_curve = self.raw_voltage - self.offset_value
```

1. Identifica el índice del valor máximo absoluto de la señal. Y toma las muestras hasta dicho índice para analizar el *pre-trigger*.

2. Extrae el 30 % inicial de las muestras del frente de onda para calcular su media aritmética y desviación estándar.
3. Identifica la muestra exacta donde la señal supera el ruido de fondo, mediante un doble análisis, primero busca hacia adelante la primera muestra que supere el umbral de 5 desviaciones estándar (determinado empíricamente usando señales reales capturadas en el laboratorio), luego busca hacia atrás la última muestra inferior a 1 desvío estándar.
4. Aísla el vector de ruido de fondo puro (`background_noise`), calcula su valor medio y lo resta a la totalidad de la curva cruda para generar el vector compensado en *offset* (`zeroed_curve`).

Método `_polarity_normalization`

Analiza la polaridad de la onda de impulso, y genera un vector equivalente positivo para simplificar el procesamiento matemático posterior.

```
def _polarity_normalization(self):
    if self.zeroed_curve is None:
        raise ValueError("Error: ...")
    self.Ue = self.zeroed_curve[self.idx_peak]
    if self.Ue >= 0:
        self.polarity = "Positiva"
        self.factor = 1.0
    else:
        self.polarity = "Negativa"
        self.factor = -1.0
    self.zeroed_curve_abs = self.zeroed_curve * self.factor
    self.peak_value = self.zeroed_curve_abs[self.idx_peak]
```

1. Valida que la curva compensada en *offset* haya sido calculada previamente.
2. Evalúa el signo de la amplitud máxima en la curva sin *offset* ($\hat{U}_0$), y define un factor multiplicador (1,0 o -1,0) según la polaridad detectada.
3. Multiplica la curva compensada en *offset* por dicho factor para obtener el vector positivo `zeroed_curve_abs` y extrae el valor extremo, U_e .

Método `_normalize_waveform`

Escala la magnitud vertical del vector positivo al rango paramétrico de 0 a 1, acorde a la recomendación de la norma IEC 60060-1 [X], ya que esto mejora la convergencia del algoritmo de ajuste para obtener la curva base.


```
def _normalize_waveform(self):
    if self.zeroed_curve_abs is None:
        raise ValueError("Error: ...")
    self.norm_voltage = self.zeroed_curve_abs / self.peak_value
```

1. Valida que la curva compensada en *offset* positiva haya sido calculada previamente.
2. Divide todos los elementos del vector `zeroed_curve_abs` por el valor extremo, U_e .
3. Almacena el resultado en el atributo `norm_voltage`.

Método `_find_limit_index`

Método estático de búsqueda de índices diseñado para identificar la intersección de la señal con un umbral de tensión específico, buscando en direcciones opuestas según la región de la onda (frente o cola).

```
@staticmethod
def _find_limit_index(v_array, threshold, mode):
    if mode == "front":
        front_reversed = v_array[::-1]
        idx_reversed = np.argmax(front_reversed < threshold)
        if idx_reversed == 0 and front_reversed[0] >= threshold:
            raise ValueError("Error: ...")
        idx = (len(v_array) - 1) - idx_reversed
    elif mode == "tail":
        idx = np.argmax(v_array < threshold)
        if not np.any(v_array < threshold):
            raise ValueError("Error: ...")
    else:
        raise ValueError("Modo desconocido.")
    return idx
```

- En modo *front*:
 1. Invierte el orden del arreglo, para descender desde el pico hacia el inicio de la onda.
 2. Busca la primera muestra que caiga por debajo del umbral, obteniendo el índice invertido.
 3. Corrige el índice al sistema de referencia original.
- En modo *tail*:
 1. Busca la primera muestra inferior al umbral, desde el pico hacia adelante.

- Implementa manejo de excepciones en caso de que la señal no alcance los umbrales exigidos, o que no se indique correctamente el modo de búsqueda.

Método `_cutting_signal`

Segmenta la curva de impulso registrada, aislando exclusivamente la región requerida para luego aplicar la regresión matemática.

```
def _cutting_signal(self):
    if self.impulse_type == "chopped":
        raise RuntimeError("Error: ...")
    if self.peak_value is None:
        raise ValueError("Error: ...")
    front_data = self.zeroed_curve_abs[:self.idx_peak]
    tail_data = self.zeroed_curve_abs[self.idx_peak:]
    threshold_20 = 0.2 * self.peak_value
    threshold_40 = 0.4 * self.peak_value
    try:
        idx_20 = self._find_limit_index(front_data, threshold_20, mode="front")
    except ValueError:
        raise ValueError("Error: ...")
    try:
        idx_40_local = self._find_limit_index(tail_data, threshold_40, mode="tail")
    except ValueError:
        raise ValueError("Error: ...")
    idx_40 = self.idx_peak + idx_40_local
    self.start_slice = idx_20 + 1
    self.end_slice = idx_40 + 1
    self.fit_voltage = self.zeroed_curve_abs[self.start_slice:self.end_slice]
    self.fit_time = self.time_axis[self.start_slice:self.end_slice]
```

1. Valida que el impulso no sea del tipo cortado, ya que este método es exclusivo para impulsos completos, y que el valor extremo de la señal haya sido calculado previamente.
2. Define los vectores de tensión correspondientes al frente y a la cola, segmentando en función del índice del valor extremo.
3. Calcula los umbrales del 20 % y el 40 % de la amplitud máxima, para luego localizar los índices correspondientes a dichos niveles en el frente y la cola, respectivamente. Implementa manejo de excepciones en caso de que la señal no alcance los umbrales exigidos, o que no se indique correctamente el modo de búsqueda.
4. Corrige el índice del 40 % al sistema de referencia original.
5. Almacena los índices de corte y genera los vectores recortados de tensión y tiempo, `.fit_voltage` y `.fit_time` respectivamente, para el ajuste de la curva base.

Método `_double_exponential_func`

Método estático que define el modelo matemático no lineal de la curva base de tensión, de acuerdo a la ecuación paramétrica de doble exponencial indicada en la norma IEC 60060-1.

```
@staticmethod
def _double_exponential_func(t, U, tau1, tau2, td):
    dt = t - td
    mask = dt >= 0
    result = np.zeros_like(dt, dtype=np.float64)
    valid_dt = dt[mask]
    val = U * (np.exp(-valid_dt / tau1) - np.exp(-valid_dt / tau2))
    result[mask] = val
    return result
```

- Recibe el vector de tiempo (t) y la estimación inicial de los parámetros de ajuste (U, τ_1, τ_2, t_d).
- Implementa una máscara lógica ($dt \geq 0$) para evitar divergencias matemáticas y números complejos en la función exponencial.
- Crea un arreglo lleno de ceros con la misma longitud que el vector dt , con tipo de datos de coma flotante de precisión doble (64 bit).
- Aplica la máscara para calcular la función solo en los puntos válidos.
- Evalúa la ecuación y retorna el vector numérico resultante, dando cero en los puntos no válidos.

Método `_fit_base_curve`

Ejecuta el algoritmo de regresión no lineal de optimización iterativa para encontrar los parámetros óptimos de la doble exponencial que mejor representen la curva base.

```
def _fit_base_curve(self):
    if self.impulse_type == "chopped":
        raise RuntimeError("Error: ...")
    if self.fit_voltage is None or self.fit_time is None:
        raise ValueError("Error: ...")
    p0_U = self.peak_value
    p0_tau1 = 70e-6
    p0_tau2 = 0.4e-6
    p0_td = self.fit_time[0]
    initial_guess = [p0_U, p0_tau1, p0_tau2, p0_td]
    idx_peak_in_slice = np.argmax(np.abs(self.fit_voltage))
    sigma = np.ones_like(self.fit_voltage)
    sigma[idx_peak_in_slice + 5] = self.sigma_fit
    try:
```

```
popt, pcov = curve_fit(
    self._double_exponential_func,
    self.fit_time,
    self.fit_voltage,
    p0 = initial_guess,
    sigma = sigma,
    absolute_sigma = False,
    maxfev = 100000)
self.fitted_params = {
    'U': popt[0],
    'tau1': popt[1],
    'tau2': popt[2],
    'td': popt[3]}
except RuntimeError as e:
    raise ValueError("Error: ...")
```

1. Valida que el impulso no sea del tipo cortado, ya que este método es exclusivo para impulsos completos, y que los vectores recortados de tensión y tiempo hayan sido calculados previamente.
2. Define las semillas iniciales (`initial_guess`) para el ajuste de los parámetros de la función doble exponencial.
3. Construye un vector de pesos (`sigma`), asignando el factor `sigma_fit` en el frente de la onda hasta el pico para forzar al algoritmo a reducir el error de ajuste en dicha zona crítica. Esto es útil cuando la señal crece muy rápido ya que el algoritmo tiene a priorizar la zona de la cola, por tener una mayor cantidad de muestras.
4. Ejecuta la función `scipy.optimize.curve_fit`, implementando el algoritmo de Levenberg-Marquardt, para obtener los parámetros óptimos que mejor ajustan la curva base.
5. Almacena los parámetros óptimos convergentes en el diccionario `fitted_params`.

Método `_construct_base_curve`

Reconstruye la curva base analítica completa ($u_m(t)$) con los parámetros óptimos encontrados en el paso anterior.

```
def _construct_base_curve(self):
    if self.impulse_type == "chopped":
        raise RuntimeError("Error: ...")
    if self.fitted_params is None:
        raise ValueError("Error: ...")
    self.base_curve = self._double_exponential_func(self.time_axis,
                                                    self.fitted_params['U'],
                                                    self.fitted_params['tau1'],
```

```
self.fitted_params['tau2'],
self.fitted_params['td'])

self.Ub = np.max(self.base_curve)
```

1. Valida que el impulso no sea del tipo cortado, ya que este método es exclusivo para impulsos completos, y que los parámetros óptimos de ajuste hayan sido calculados previamente.
2. Genera la curva base completa evaluando la función de doble exponencial con los parámetros óptimos sobre el eje temporal original completo.
3. Extrae y almacena el valor máximo absoluto de la curva base ajustada (Ub).

Método `_calculate_residual_curve`

Calcula la curva residual que aísla la sobreoscilación y el ruido de alta frecuencia inherentes a la medición.

```
def _calculate_residual_curve(self):
    if self.zeroed_curve_abs is None:
        raise ValueError("Error: ...")
    if self.base_curve is None:
        raise ValueError("Error: ...")
    self.residual_curve = self.zeroed_curve_abs - self.base_curve
```

1. Valida que la curva compensada en *offset* positiva y la curva base ajustada, hayan sido calculadas previamente.
2. Calcula la curva residual restando la curva base a la curva compensada en *offset* positiva.
3. Almacena el resultado en el atributo `residual_curve`.

Método `_create_digital_filter`

Sintetiza los coeficientes para un filtro digital de respuesta impulsional infinita (IIR) diseñado según la ecuación de diferencia expuestas en la ecuación 1.6.

```
def _create_digital_filter(self):
    d = 2.2e-12
    c = np.tan((np.pi * self.sampling_period) / np.sqrt(d))
    b0 = c / (1 + c)
    b1 = b0
    a1 = (1 - c) / (1 + c)
    b = np.array([b0, b1])
    a = np.array([1.0, -a1])
    return b, a
```

1. Inicializa la constante del filtro: $d = 2,2 \times 10^{-12}$.
2. Calcula la constante intermedia (c) basada en el periodo de muestreo.
3. Determina los coeficientes analíticos de transferencia b_0 , b_1 y a_1 , retornando las matrices de numerador y denominador requeridas por SciPy.

Método `_filter_to_residual`

Aplica el filtro digital definido en la ecuación 1.6 a la curva residual aislada en el paso anterior.

```
def _filter_to_residual(self):
    if self.residual_curve is None:
        raise ValueError("Error: ...")
    b, a = self._create_digital_filter()
    self.filtered_residual = signal.filtfilt(b, a, self.residual_curve)
```

1. Valida que la curva residual haya sido calculada previamente.
2. Obtiene los coeficientes del filtro IIR.
3. Implementa un filtrado bidireccional (*zero-phase filtering*) mediante la función `scipy.signal.filtfilt`, anulando el retraso de fase de la curva residual.

Método `_construct_test_voltage_curve`

Sintetiza la curva de tensión de ensayo definitiva sobre la cual se calcularán los parámetros temporales y de amplitud, de la onda de impulso.

```
def _construct_test_voltage_curve(self):
    if self.base_curve is None:
        raise ValueError("Error: ...")
    if self.filtered_residual is None:
        raise ValueError("Error: ...")
    self.test_voltage_curve_abs = self.base_curve + self.filtered_residual
    self.Ut = np.max(self.test_voltage_curve_abs) * self.factor
    self.test_voltage_curve = self.test_voltage_curve_abs * self.factor
    self.results["Ut"] = self.Ut
    self.test_voltage_curve_norm = self.test_voltage_curve / np.abs(self.Ut)
```

1. Valida que la curva base y la curva residual filtrada hayan sido calculadas previamente.
2. Suma la curva base con la curva residual filtrada.
3. Restaura el signo original de la onda aplicando el factor de polaridad.

4. Extrae el valor cresta de la curva de ensayo (U_t) y calcula una versión normalizada de la curva de ensayo.

Método `_linear_interpolation`

Método estático que implementa el algoritmo de interpolación lineal para encontrar con mayor precisión el cruce de umbrales sobre el eje de tiempo.

```
@staticmethod
def _linear_interpolation(t_array, v_array, idx_low, target_voltage):
    v1 = v_array[idx_low]
    t1 = t_array[idx_low]
    if idx_low + 1 < len(v_array) and v_array[idx_low + 1] > v1:
        idx_high = idx_low + 1
    elif idx_low - 1 >= 0 and v_array[idx_low - 1] > v1:
        idx_high = idx_low - 1
    else:
        return t1
    v2 = v_array[idx_high]
    t2 = t_array[idx_high]
    return t1 + (target_voltage - v1) * ((t2 - t1) / (v2 - v1))
```

1. Determina el par de coordenadas bidimensional de las muestra pasada como referencia.
2. Busca si el umbral objetivo es anterior o posterior a la referencia.
3. Calcula el par de coordenadas bidimensional del segundo punto para determinar la recta que pasa por el umbral objetivo.
4. Evalúa la ecuación de la recta para calcular y retornar el instante de tiempo correspondiente al umbral buscado.

Método `_calc_front_parameters`

Determina el tiempo de frente y el origen virtual en la zona de ascenso de la onda.

```
def _calc_front_parameters(self, Ut):
    front_v = self.test_voltage_curve_abs[:self.idx_peak_Ut]
    front_t = self.time_axis[:self.idx_peak_Ut]
    v30 = 0.3 * Ut
    v90 = 0.9 * Ut
    idx_30 = self._find_limit_index(front_v, v30, mode="front")
    idx_90 = self._find_limit_index(front_v, v90, mode="front")
    t30 = self._linear_interpolation(front_t, front_v, idx_30, v30)
    t90 = self._linear_interpolation(front_t, front_v, idx_90, v90)
    Tab = t90 - t30
    T1 = Tab / 0.6
```

```
O1 = t30 - 0.5 * Tab
return O1, T1
```

1. Localiza los índices discretos correspondientes a los umbrales del 30 % y 90 % de U_t .
2. Interpola los tiempos exactos t_{30} y t_{90} .
3. Calcula el tiempo de frente (T_1).
4. Proyecta el origen virtual (O_1) extrapolando linealmente la recta de frente hasta tensión nula.

Método `_calc_tail_parameter`

Determina el tiempo de cola en la zona de descenso de la onda.

```
def _calc_tail_parameter(self, Ut, O1):
    tail_v = self.test_voltage_curve_abs[self.idx_peak_Ut:]
    tail_t = self.time_axis[self.idx_peak_Ut:]
    v50 = 0.5 * Ut
    idx_50 = self._find_limit_index(tail_v, v50, mode="tail")
    t50 = self._linear_interpolation(tail_t, tail_v, idx_50, v50)
    T2 = t50 - O1
    return T2
```

1. Localiza el índice discreto y el tiempo interpolado correspondiente a la caída del 50 % de U_t (t_{50}).
2. Resta el origen virtual O_1 al instante t_{50} calculado para obtener el valor de T_2 .

Método `_calculate_parameters`

Dirige el cálculo de los parámetros de la onda de impulso (U_t , T_1 , T_2 y OS).

```
def _calculate_parameters(self):
    if self.test_voltage_curve_abs is None:
        raise ValueError("Error: ...")
    Ut = np.abs(self.Ut)
    self.idx_peak_Ut = np.argmax(self.test_voltage_curve_abs)
    self.results["OS"] = 100 * (self.peak_value - self.Ub) / self.peak_value
    try:
        O1, T1 = self._calc_front_parameters(Ut)
        self.results["T1"] = T1
    except ValueError:
        raise ValueError("Error: ...")
    try:
```



```
if self.impulse_type == "full":
    T2 = self._calc_tail_parameter(Ut, 01)
elif self.impulse_type == "chopped":
    T2 = self.Tcutting_moment - 01
else:
    raise ValueError("Error: ...")
self.results["T2"] = T2
except ValueError:
    raise ValueError("Error: ...")
```

1. Valida que la curva de tensión de ensayo positiva haya sido calculada previamente.
2. Extrae el instante del pico máximo de la tensión de ensayo.
3. Calcula la sobreoscilación de alta frecuencia (*OS*) comparando el valor máximo de la curva de tensión de ensayo y de la curva base.
4. Ejecuta los métodos de cálculo de frente y evalúa la variable `impulse_type` para bifurcar la lógica de cálculo del tiempo de cola, dependiendo de si el transitorio decae de forma natural o fue cortado por una descarga disruptiva.

Método `_find_time_lag`

Analiza la desviación temporal entre la última onda captura y una onda completa de referencia capturada inicialmente.

```
def _find_time_lag(self, ref_analyzer):
    levels = [0.3, 0.5, 0.8]
    t_diffs = []
    for level in levels:
        front_self = self.norm_voltage[:self.idx_peak]
        idx_self = self._find_limit_index(front_self, level, mode="front")
        t_self = self._linear_interpolation(self.time_axis, self.norm_voltage,
                                           idx_self, level)

        front_ref = ref_analyzer.norm_voltage[:ref_analyzer.idx_peak]
        idx_ref = self._find_limit_index(front_ref, level, mode="front")
        t_ref = self._linear_interpolation(ref_analyzer.time_axis, ref_analyzer.
                                           norm_voltage, idx_ref, level)

        t_diffs.append(t_ref - t_self)
    self.t_L = np.mean(t_diffs)
```

1. Interpola los instantes exactos en los que ambas ondas normalizadas (actual y referencia) cruzan los umbrales del 30 %, 50 % y 80 %.
2. Calcula la diferencia de tiempo punto a punto y determina el desfase medio (t_L).

Método `_adjust_time_lag`

Desplaza el eje temporal de la última onda capturada para sincronizar su frente de onda con el impulso de referencia.

```
def _adjust_time_lag(self):
    if self.t_L is None:
        raise ValueError("Error: ...")
    self.aligned_time_axis = self.time_axis + self.t_L
```

1. Valida que desfase medio (t_L) haya sido calculada previamente.
2. Suma escalarmente el desfase calculado (t_L) al vector de eje de tiempo original.

Método `_find_deviation_point`

[Específico para impulsos cortados] Detecta algorítmicamente el punto en el cual la onda evaluada colapsa antes de tiempo debido a una descarga disruptiva.

```
def _find_deviation_point(self, ref_analyzer, threshold=0.02):
    if self.norm_voltage is None or ref_analyzer.norm_voltage is None:
        raise ValueError("Error: ...")
    if self.aligned_time_axis is None:
        raise ValueError("Error: ...")
    chopped_norm_interp = np.interp(
        ref_analyzer.time_axis,
        self.aligned_time_axis,
        self.norm_voltage,
        left=0.0, right=0.0)
    tail_self = chopped_norm_interp[ref_analyzer.idx_peak:]
    tail_ref = ref_analyzer.norm_voltage[ref_analyzer.idx_peak:]
    diff = np.abs(tail_self - tail_ref)
    mask_dev = diff > threshold
    if mask_dev.any():
        self.impulse_type = "chopped"
    else:
        self.impulse_type = "full"
    return
    idx_dev_local = np.argmax(mask_dev)
    self.idx_deviation = ref_analyzer.idx_peak + idx_dev_local
```

1. Valida que las ondas normalizadas (actual y referencia) hayan sido calculadas previamente, y que el tiempo de la onda actual haya sido alineado con la referencia.
2. Interpola la señal analizada mapeándola sobre el eje temporal sincronizado de la señal de referencia.

3. Calcula la diferencia punto a punto en la región posterior al pico y la evalúa contra un umbral de tolerancia del 2 % (hay que aumentarlo).
4. Actualiza la bandera de estado, indicando si la onda es completa o cortada.
5. Almacena el índice exacto de la bifurcación.

Método `_select_data_up_to_deviation`

[Específico para impulsos cortados] Purga la información de alta frecuencia posterior al corte del aislamiento.

```
def _select_data_up_to_deviation(self):
    if self.idx_deviation is None:
        raise ValueError("Error: ...")
    idx_end = self.idx_deviation + 1
    self.chopped_front_voltage = self.norm_voltage[:idx_end].copy()
    self.chopped_front_time = self.time_axis[:idx_end].copy()
```

1. Valida que índice de la bifurcación haya sido calculado previamente.
2. Realiza un seccionamiento de los vectores de tensión y tiempo utilizando el índice de desviación.
3. Realiza copias de los segmentos para aislar los vectores originales de cualquier alteración.

Método `_find_amplitude_ratio`

[Específico para impulsos cortados] Determina la relación de escala en amplitud entre el ensayo a tensión plena (cortado) y el ensayo a tensión reducida (referencia).

```
def _find_amplitude_ratio(self, ref_analyzer):
    if self.aligned_time_axis is None:
        raise ValueError("Error: ...")
    front_self_norm = self.norm_voltage[:self.idx_peak]
    idx_30 = self._find_limit_index(front_self_norm, 0.3, mode="front")
    idx_80 = self._find_limit_index(front_self_norm, 0.8, mode="front")
    vals_self = self.zeroed_curve_abs[idx_30:idx_80]
    t_interval = self.aligned_time_axis[idx_30:idx_80]
    vals_ref = np.interp(
        t_interval,
        ref_analyzer.time_axis,
        ref_analyzer.zeroed_curve_abs)
    self.E = np.mean(vals_self) / np.mean(vals_ref)
```

1. Valida que el eje de tiempo alineado haya sido calculado previamente.

2. Promedia la tensión real contenida en el intervalo de mayor linealidad (entre el 30 % y el 80 % del frente).
3. Calcula la división entre la media de la señal bajo ensayo y la media de la señal de referencia, obteniendo el escalar E .

Método `_scale_base_curve`

[Específico para impulsos cortados] Evita el algoritmo de Levenberg-Marquardt al no disponer de una cola de onda válida, generando la curva teórica a partir del modelo de la onda de referencia.

```
def _scale_base_curve(self, ref_analyzer):
    U_ref = ref_analyzer.fitted_params['U']
    tau1 = ref_analyzer.fitted_params['tau1']
    tau2 = ref_analyzer.fitted_params['tau2']
    td = ref_analyzer.fitted_params['td']
    U_scaled = U_ref * self.E
    self.base_curve = self._double_exponential_func(
        self.aligned_time_axis,
        U_scaled,
        tau1,
        tau2,
        td)
    self.Ub = np.max(self.base_curve)
```

1. Extrae los parámetros de ajuste (U , τ_1 , τ_2 , t_d) de la onda de referencia.
2. Multiplica la amplitud máxima del impulso de referencia (U_{ref}) por el factor de escala (E), para que tenga la misma magnitud que la onda a evaluar.
3. Reconstruye la curva base del impulso cortado evaluando la ecuación exponencial con los parámetros de la referencia.

Método `_find_chopping_instant`

[Específico para impulsos cortados] Estima el instante analítico del corte del impulso de forma análoga al cálculo del tiempo de frente de un impulso estándar.

```
def _find_chopping_instant(self):
    if self.norm_voltage is None:
        raise ValueError("Error: ...")
    tail_vals = self.norm_voltage[self.idx_peak:]
    gradient = np.gradient(tail_vals)
    idx_steepest_local = np.argmin(gradient)
    grad_2 = np.gradient(gradient[idx_steepest_local])
    idx_knee_local = np.argmin(grad_2)
```

```
idx_collapse = self.idx_peak + idx_knee_local
U_collapse = self.zeroed_curve_abs[idx_collapse]
v70 = 0.7 * U_collapse
v10 = 0.1 * U_collapse
fall_segment = self.zeroed_curve_abs[idx_collapse:]
idx_70_local = np.argmax(fall_segment < v70)
idx_10_local = np.argmax(fall_segment < v10)
if idx_70_local == 0 or idx_10_local == 0:
    raise ValueError("Error: ...")
idx_C = idx_collapse + idx_70_local
idx_D = idx_collapse + idx_10_local
t_C, v_C = self.time_axis[idx_C], self.zeroed_curve_abs[idx_C]
t_D, v_D = self.time_axis[idx_D], self.zeroed_curve_abs[idx_D]
if t_D == t_C:
    raise ValueError("Error: ...")
slope = (v_D - v_C) / (t_D - t_C)
self.Tcutting_moment = t_C + (U_collapse - v_C) / slope
self.U_collapse = U_collapse
```

1. Valida que la versión normalizada de la onda bajo análisis haya sido calculada previamente.
2. Calcula el gradiente de los valores posteriores al pico para encontrar el punto de máxima pendiente negativa, donde se produjo el colapso de la onda.
3. Determina la tensión inmediatamente antes del colapso, usando el mínimo de la segunda derivada.
4. Calcula los valores del 70 % y el 10 % referidos la tensión de colapso, y determina los índices locales de las muestras discretas inmediatamente inferiores.
5. Corrige los índices al sistema de referencia original.
6. Obtiene el par de coordenadas bidimensional del 70 % y el 10 % de la tensión de colapso, para para extrapolar el momento virtual de corte (Tcutting_moment).

Método `ref_lightning_impulse`

Ejecuta de manera secuencial los métodos privados de procesamiento integral para calcular los parámetros de una onda de impulso completo a tensión reducida (referencia).

```
def ref_lightning_impulse(self):
    self.impulse_type = "full"
    # Ejecutar pipeline completo.
    self._remove_offset()
    self._polarity_normalization()
    self._normalize_waveform()
    #-----
```

```

self._cutting_signal()
self._fit_base_curve()
self._construct_base_curve()
#-----
self._calculate_residual_curve()
self._filter_to_residual()
self._construct_test_voltage_curve()
self._calculate_parameters()

```

Método `lightning_impulse`

Orquesta el *pipeline* de procesamiento adaptativo diseñado para transitorios capturados durante el ensayo a tensión plena de la máquina o aislador.

Ejecuta de manera secuencial los métodos privados de procesamiento adaptativo para calcular los parámetros de los impulsos capturados durante el ensayo a tensión plena (completo o cortados).

Requiere y valida la inyección de la instancia del analizador de referencia calculada previamente.

```

def lightning_impulse(self, ref_analyzer):
    if ref_analyzer is None:
        raise ValueError("Error: ...")
    self._remove_offset()
    self._polarity_normalization()
    self._normalize_waveform()
    #-----
    self._find_time_lag(ref_analyzer)
    self._adjust_time_lag()
    self._find_deviation_point(ref_analyzer)
    if self.impulse_type == "full":
        self._cutting_signal()
        self._fit_base_curve()
        self._construct_base_curve()
    elif self.impulse_type == "chopped":
        self._select_data_up_to_deviation()
        self._find_amplitude_ratio(ref_analyzer)
        self._scale_base_curve(ref_analyzer)
        self._find_chopping_instant()
    #-----
    self._calculate_residual_curve()
    self._filter_to_residual()
    self._construct_test_voltage_curve()
    self._calculate_parameters()

```